

Documentazione Progetto Reti Informatiche

Ingegneria Informatica UNIPI A.A. 25/26 (DISPARI)

January 2, 2026

Luca Serafini (672707)

Descrizione Kanban è un'applicazione utilizzata per gestire una lavagna Kanban e gli utenti associati ad essa. L'applicazione presenta un'architettura ibrida: di tipo *Client-Server* per il collegamento tra utente e lavagna, data la necessità per gli utenti di riferirsi ad un unico dispositivo per la gestione delle card; *Peer-to-Peer* per il collegamento tra utenti, che si inviano messaggi sporadici per la revisione dei loro job.

Struttura del progetto Il progetto è suddiviso in varie cartelle: la cartella *class* è condivisa tra lavagna e utenti, contiene strutture dati e funzioni relative ad esse; vi è poi il file *structs/enums.h* che contiene tipi enumerati e macro; *network* per implementare funzionalità di rete; *functions* e *server*, cartelle che implementano funzionalità relative alla sola lavagna e user, che contiene le funzioni relative all'utente.

Lavagna La **lavagna** è l'elemento centrale dell'architettura. Essa sfrutta l'IO-multiplexing, tramite la chiamata di sistema *select*, per rimanere in ascolto di:

- *Socket* TCP degli utenti
- Una pipe, utilizzata per la gestione del timer

Essendo l'applicazione a comunicazione asincrona, e supponendo un numero ridotto di utenti¹, ho scelto di utilizzare la *select*, con un solo thread, dato che le singole operazioni sono molto rapide, e non richiedono elevati tempi di computazione. Nonostante una soluzione multi-thread possa risultare più rapida, dato il tipo di applicazione non è necessaria una velocità così elevata, ed è quindi più conveniente utilizzare la soluzione single-thread, evitando possibili errori nell'implementazione della *mutua esclusione*. Una volta ricevuto un segnale dall'utente, si verifica se sia un tentativo di connessione o un messaggio socket²: nel primo caso si verifica che non esista un altro utente con lo stesso numero di porta, in tal caso si crea l'utente e gli si fornisce la card; nel secondo si passa il comando alla funzione *handle_message*, che esegue le operazioni necessarie. Il server utilizza una struttura

dati di tipo **Board** per gestire gli utenti connessi e le card.

Timer Non essendo specificato nella richiesta, ho ritenuto che se un utente non risponde con *ACK_CARD* ad una *HANDLE_CARD* del server per più di 120 secondi, esso viene considerato inattivo e conseguentemente disconnesso. Per l'implementazione di questa funzionalità, quella della *PING_LAVAGNA* e quella di disconnessione dell'utente alla mancata *PONG_LAVAGNA*, ho creato una struttura dati *Timer_s*, ovvero una lista che ad ogni elemento contiene: una funzione e il suo argomento (porta dell'utente), la porta dell'utente e il timestamp a cui dev'essere eseguita. L'inserimento degli elementi nel timer avviene sempre in ordine di timestamp, in modo tale che gli eventi più vicini siano in testa. Successivamente si chiama la *alarm* sul tempo che intercorre il timestamp attuale e l'elemento in testa della lista, al termine del quale viene segnalata alla *select* che una funzione è pronta per essere eseguita (scrivendo un valore su una pipe attenzionata dalla *select*), il quale la eseguirà, e nel caso vi siano altri elementi in lista, verrà chiamata un *alarm* sull'elemento in testa. L'uso di *alarm* è adeguato data la presenza di un singolo thread e di un solo flusso di gestione degli eventi temporali.

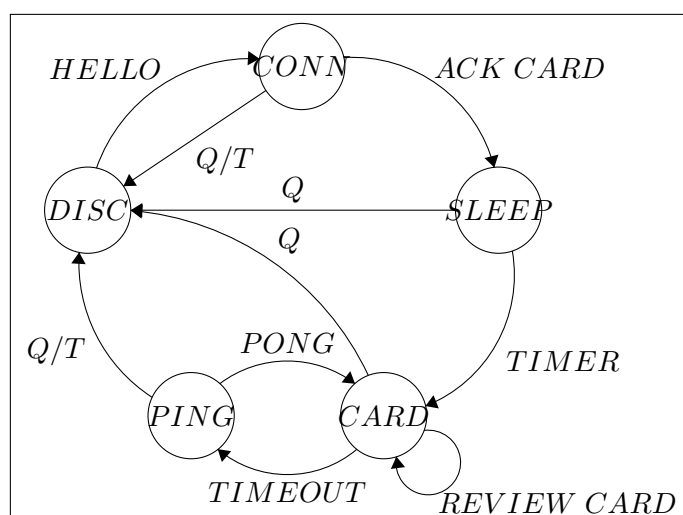


Fig. 1: Automa a stati finiti (FSM) dell'Utente

¹ Si presuppone che gli utilizzatori di questa applicazione siano team di dimensioni modeste

² Un messaggio che arriva alla *select* da un utente è sempre un comando

Utenti Gli utenti rappresentano i vari client del progetto. Gli utenti possono essere definiti tramite un au-

toma a stati finiti (Fig. 1). Nel mio caso ho implementato questo concetto tramite l'ausilio del tipo enumerato **User_status** (*enums.h*). Esso identifica lo stato corrente dell'utente, e in base a ciò viene permesso o meno all'utente di avviare date funzioni in base allo stato. Per gestire l'utente vengono utilizzate due variabili: **user_data** e **review**. La prima contiene tutte le informazioni necessarie relative all'utente: stato in cui si trova, carta che sta gestendo, porte e socket della lavagna e degli altri peer. La seconda, invece, viene utilizzata durante il processo di review, e contiene un array con gli utenti che devono ancora revisionare la propria card. L'utente utilizza una select in ascolto del socket TCP con la lavagna, UDP con gli altri utenti e una pipe per gli eventi del timer. Per simulare lo svolgimento della card, si utilizza un altro thread, che chiama la sleep per un tempo casuale compreso tra i 10 e i 30 secondi.

Comunicazione Utente-Lavagna La comunicazione tra utente e lavagna avviene tramite un socket TCP, seguendo il paradigma Client-Server, dato che l'utente si registra fisicamente alla lavagna e invia spesso comandi. I comandi e i dati passati, devono arrivare tutti integri, per questo ho scelto un protocollo di comunicazione affidabile.

Comunicazione tra utenti La comunicazione tra utenti avviene tramite un socket UDP. Ogni utente è in ascolto del suo socket, e invia messaggi agli altri, sapendo che il loro socket si trova alla porta da loro specificata come ID. La comunicazione Peer-to-Peer avviene esclusivamente per la revisione di card, per questo utilizzo il protocollo non affidabile: essendo una sorta di "notifica", non è necessario che gli utenti siano tutti connessi³. Per questa funzionalità ho deciso di utilizzare questo paradigma, poiché riflette la natura logica dell'operazione: mentre la Lavagna rappresenta l'autorità centrale per la persistenza dei dati, la revisione è un'interazione diretta e transitoria tra due pari. La revisione avviene tramite l'invio del comando **REVIEW_CARD**, con il quale si invia agli altri utenti attivi una notifica relativa all'utente che richiede la revisione. Nel frattempo viene avviato un timer di 30 secondi. Agli utenti che devono revisionare viene aggiunto un comando **REVIEW**, tramite il quale possono revisionare il task dell'utente che hanno ricevuto per primo⁴. Nel caso in cui il messaggio UDP o la notifica di revisione

non arrivasse all'altro host, il timer, ogni 30 secondi, controlla gli utenti della lavagna rimasti (se ne sono stati rimossi alcuni, essi verranno rimossi anche dalla struttura "review", nel caso ne venissero aggiunti altri rispetto allo stato iniziale, essi non revisioneranno la card), inviandogli un'altra notifica.

Protocolli di invio dei messaggi Per inviare i messaggi ho scelto di utilizzare sia protocolli text che binary. Distinguiamo le principali macro-categorie di messaggi:

- I **comandi TCP**, ovvero quelli tra lavagna e utente, sono gestiti tramite due strutture di tipo enumerato, **Board_to_User_command** (per inviare messaggi dalla lavagna all'utente) e **User_to_Board_command** (per l'operazione inversa). La scelta ricade quindi sul protocollo binary. Questa scelta garantisce efficienza (dimensione del pacchetto fissa e ridotta) e scalabilità, permettendo l'aggiunta di nuove funzionalità semplicemente estendendo la definizione dell'enumerato, senza modificare la logica di parsing di basso livello.
- I **comandi UDP**, ovvero quelli che vengono inviati tra utenti, utilizzano anch'essi il protocollo binary, dato che ho bisogno di una struttura ben definita che mi specifichi porta dell'utente trasmettitore e carta che deve essere revisionata (o -1, nel caso di messaggio di revisione)⁵.
- Il resto dei messaggi varia: per inviare i numeri utilizzo il protocollo binario; le **stringhe di testo**, invece, vengono inviate precedute da un intero che ne indica la dimensione esatta. Questo permette al ricevente di allocare dinamicamente il buffer corretto e leggere esattamente i byte necessari, evitando le ambiguità tipiche dei protocolli puramente testuali basati su delimitatori.

Istruzioni di compilazione Per compilare i file è sufficiente utilizzare il comando

```
make
```

³ Questo implica anche meno tunnel TCP da gestire

⁴ Dato che possono essere inviati più task da più utenti diversi, i successivi possono essere revisionati in ordine FCFS

⁵ L'uso di una struct a dimensione fissa evita l'overhead di parsing complesso.